

SeamlessM4T v2 Large: 2.3B → ~1B

Compression Plan

Diagnosis: Why Your Previous Approach Failed

Problem 1: You only pruned the speech encoder — it's NOT the largest component

SeamlessM4T v2 Large parameter breakdown (approximate):

Component	Params	% of total
Speech Encoder (Conformer-Shaw)	~635M	~28%
Text Encoder (NLLB 24 layers)	~350M*	~15%
Text Decoder (NLLB 24 layers)	~350M*	~15%
Shared Embeddings (256,102 vocab)	~262M	~11%
T2U Model (NAR, 6+6 layers)	~150-200M	~8%
Other (adaptors, projections)	~100M	~4%
TOTAL (excl. vocoder)	~2,300M	
Vocoder (CodeHiFiGAN, separate)	~100M	separate

* Text encoder/decoder share embedding weights with the 256K vocabulary

Your notebook pruned only the speech encoder (635M). Even removing 40% of it only saves ~240M → a 10% total reduction. The IWSLT 2025 paper explicitly showed that pruning decoder layers is **MORE effective** than encoder layers.

Problem 2: Magnitude-based FFN pruning is the worst modern pruning metric

Your code used:

```
scores = weight.abs().mean(dim=1) # L1 norm of weight rows
```

This is pure **magnitude pruning** — known since 2023 to be far inferior to activation-aware methods. Modern alternatives:

Method	Metric	Performance
Magnitude (yours)	`	W
Wanda (ICLR 2024)	`	W
FLAP (AAAI 2024)	Fluctuation of output features	Best retraining-free structured
SliceGPT (ICLR 2024)	Orthogonal projection PCA	True width reduction

Problem 3: Sparse zeros don't speed up GPU inference (explains your RTF increase)

You zeroed out neurons but left them in the weight matrices:

```
fc1.weight.data[~mask] = 0 # Still a full-size dense matrix!
```

GPUs process dense tensors. Zeros don't skip computation — they actually add overhead from the masking operations. This is why your RTF **increased** from 0.24 → 0.35. True speedup requires **structural removal** (actually deleting rows/columns from weight matrices, removing entire layers, or reducing vocabulary).

Problem 4: Layer importance via angular distance is a reasonable proxy but not optimal

The IWSLT 2025 paper compared three approaches:

- 1. **Middle layer removal** (naive) → worst results
- 2. **Angular distance / Block Influence** (what you used) → moderate
- 3. **Iterative greedy pruning with task evaluation** → best by a large margin

Key finding: iterative greedy pruning found layers to remove that angular distance would have ranked differently, because layer interactions matter.

Problem 5: 256K vocabulary for 100 languages when you need ~5

The NLLB embedding matrix has **256,102 tokens × 1024 dimensions = 262M parameters**. ~80% of these tokens serve languages you don't need. This is pure waste.

Cutting-Edge Research Papers (2024-2025)

Primary Papers to Follow

#	Paper	Venue	Key Technique	Relevance
1	Moslem (2025) "Efficient Speech Translation through Model Compression and Knowledge Distillation"	IWSLT 2025	Iterative layer pruning + QLoRA + KD	Directly applicable — 50% compression of speech translation model retaining 97-100% quality
2	Moslem et al. (2025) "Iterative Layer Pruning for Efficient Translation Inference"	WMT 2025	Greedy layer pruning guided by chrF/chrF++	Directly applicable — iterative is far superior to one-shot
3	Rostami & Dousti (2024) "CULL-MT: Compression Using Language and Layer pruning"	arXiv 2411.06506	Language-specific layer pruning on NLLB-3.3B	Directly applicable — prunes NLLB (same backbone as SeamlessM4T text components), 25% layer removal with only 0.9 spBLEU drop
4	An et al. (2024) "FLAP: Fluctuation-based Adaptive Structured Pruning"	AAAI 2024	Width pruning (FFN + attention heads) without retraining	Use for FFN/attention head pruning instead of magnitude
5	Sun et al. (2024) "Wanda: A Simple and Effective	ICLR 2024	`	W

#	Paper	Venue	Key Technique	Relevance
	Pruning Approach"			
6	Ashkboos et al. (2024) "SliceGPT: Compress by Deleting Rows and Columns"	ICLR 2024	Orthogonal projection to reduce embedding dimension	Advanced option for true width reduction
7	Men et al. (2024) "ShortGPT: Layers are More Redundant Than You Expect"	ACL 2025 Findings	Block Influence metric for layer removal	Better layer importance metric than angular distance
8	Yang et al. (2024) "LaCo: Layer Collapse"	EMNLP 2024 Findings	Merge adjacent redundant layers	Alternative to pure layer deletion
9	Asahi et al. (2023) "Vocabulary Trimming for Multilingual LMs"	EMNLP 2023 Findings	Remove tokens for unused languages	Critical — can save 150-200M params from embeddings
10	Peng et al. (2023) "DPHuBERT: Joint Distillation and Pruning of Speech Models"	InterSpeech 2023	Structured pruning of speech encoders	Applicable to speech encoder specifically

The GitHub Repository to Study

ymoslem/Model-Compression (<https://github.com/ymoslem/Model-Compression>) This is the code from the IWSLT 2025 and WMT 2025 papers. It contains:

- Layer importance evaluation scripts
- Iterative layer pruning implementation

- QLoRA fine-tuning pipeline
- Knowledge distillation pipeline

Comprehensive Pruning Plan: 2.3B → ~1B

Target Budget

Starting:	2,300M params
Target:	~1,000-1,200M params
Reduction:	~1,100-1,300M params (~50% compression)

Strategy Overview

Phase 0: Baseline measurement (no changes)	
Phase 1: Vocabulary/Embedding Pruning	→ save ~200M params
Phase 2: Text Decoder Iterative Layer Pruning	→ save ~150-200M params
Phase 3: Text Encoder Layer Pruning	→ save ~100-150M params
Phase 4: Speech Encoder Layer Pruning	→ save ~150-200M params
Phase 5: Width Pruning (FLAP on FFN+heads)	→ save ~200-300M params
Phase 6: T2U Model Pruning	→ save ~50-100M params
Phase 7: Recovery Fine-tuning (S2TT CE loss)	→ recover quality
Phase 8: Final Benchmark	
TOTAL: ~850-1,250M reduction	

Phase 1: Vocabulary/Embedding Pruning (~200M reduction)

Paper: Asahi et al. (EMNLP 2023), CULL-MT (arXiv 2024)

Why this is the easiest win: SeamlessM4T uses the NLLB-200 vocabulary of 256,102 tokens covering ~100 languages. You only need ~5 languages (English, Bengali, Chinese, French, German, Hindi, Urdu). The vocabulary for these languages is likely ~30-50K tokens.

Method:

1. Identify all tokens used by your target languages using the SentencePiece tokenizer and language-specific corpora
2. Keep: all tokens that appear in your target language corpora + special tokens

- shared subword tokens
3. Remove: rows from the embedding matrix and columns from the output projection that correspond to unused tokens
 4. Rebuild the tokenizer with the trimmed vocabulary

Implementation:

```
import torch
from transformers import SeamlessM4Tv2ForSpeechToSpeech, AutoTokenizer

# Step 1: Identify tokens used by target languages
target_langs = ["eng", "ben", "cmn", "fra", "deu", "hin", "urd"]

# Load tokenizer and find language-specific tokens
tokenizer = AutoTokenizer.from_pretrained("facebook/seamless-m4t-v2-large")

# Use corpora from FLORES/FLEURS to identify which tokens are actually used
# For each target language, tokenize a large corpus and collect unique token IDs
used_token_ids = set()
used_token_ids.update(tokenizer.all_special_ids) # Always keep special tokens

for lang in target_langs:
    # Load FLORES-200 devtest for each language
    # Tokenize all sentences, collect unique token IDs
    corpus = load_flores_data(lang)
    for text in corpus:
        token_ids = tokenizer.encode(text)
        used_token_ids.update(token_ids)

# Step 2: Create mapping from old to new token IDs
old_to_new = {old_id: new_id for new_id, old_id in enumerate(sorted(used_token_ids))}
new_vocab_size = len(used_token_ids)

# Step 3: Slice embedding matrices
old_embed = model.shared.weight.data # [256102, 1024]
new_embed = old_embed[sorted(used_token_ids)] # [new_vocab_size, 1024]

# Step 4: Replace in model
model.shared = nn.Embedding(new_vocab_size, 1024)
model.shared.weight.data = new_embed
# Also update lm_head / final_proj if not tied
```

Expected savings: 256K → ~40K tokens = ~216K × 1024 × 2 (embed + output proj, if not tied) ≈ ~220M parameters

Risk: Low — tokens for unused languages don't contribute to your translation.

Phase 2: Text Decoder Iterative Layer Pruning (~150-200M reduction)

Paper: Moslem (IWSLT 2025), CULL-MT (2024)

Why the decoder: The IWSLT paper showed decoder-only pruning outperforms encoder-decoder pruning. The text decoder has 24 NLLB layers. CULL-MT showed NLLB-3.3B tolerates 25% layer removal with only 0.9 spBLEU drop.

Method (Iterative Greedy Pruning):

```
for i in range(n_layers_to_remove):
    best_layer = None
    best_score = -inf
    for candidate_layer in remaining_layers:
        temp_model = remove_layer(model, candidate_layer)
        score = evaluate_bleu_chrf(temp_model, validation_set)
        if score > best_score:
            best_score = score
            best_layer = candidate_layer
    permanently_remove(model, best_layer)
    print(f"Iteration {i}: removed layer {best_layer}, score={best_score}")
```

This is fundamentally different from your previous approach: instead of using a proxy metric (angular distance), you directly measure task performance.

Target: Remove 6-8 of 24 text decoder layers (25-33%)

Implementation:

```
def iterative_layer_pruning(model, processor, eval_dataset, n_prune, component="t
    """
    Greedy iterative layer pruning guided by actual task performance.
    Based on Moslem (IWSLT 2025) and CULL-MT (2024).
    """
    import copy

    decoder = getattr(model, component)
    remaining_indices = list(range(len(decoder.layers)))
    removed = []
```

```

for iteration in range(n_prune):
    scores = {}
    for idx in remaining_indices:
        # Temporarily remove this layer
        temp_layers = [decoder.layers[i] for i in remaining_indices if i != idx]
        original_layers = decoder.layers
        decoder.layers = torch.nn.ModuleList(temp_layers)

        # Evaluate on validation set (use BLEU or ChrF)
        score = evaluate_s2t_bleu(model, processor, eval_dataset)
        scores[idx] = score

    # Restore
    decoder.layers = original_layers

    # Remove the layer whose absence causes LEAST degradation
    best_layer = max(scores, key=scores.get)
    remaining_indices.remove(best_layer)
    removed.append(best_layer)

    # Actually remove it permanently
    decoder.layers = torch.nn.ModuleList(
        [decoder.layers[i] for i in remaining_indices]
    )
    # Re-index remaining_indices to 0..len-1
    remaining_indices = list(range(len(decoder.layers)))

    print(f"Iter {iteration+1}: removed layer {best_layer}, "
          f"score={scores[best_layer]:.2f}, remaining={len(remaining_indices)}")

return removed

```

IMPORTANT: Use ChrF/ChrF++ as the evaluation metric for layer importance (not COMET or BLEU), as the IWSLT paper found ChrF-guided pruning produces better final models.

Expected savings: ~6-8 layers × ~20M params/layer ≈ ~120-160M params

Phase 3: Text Encoder Layer Pruning (~100-150M reduction)

Paper: Same iterative method as Phase 2

The text encoder has 24 NLLB layers. For S2S translation, the speech encoder output goes through an adaptor and then the text decoder uses cross-attention to the encoder

output. The text encoder is used when text input is given (T2T, T2S tasks), but for pure S2S you may not even need it.

Critical question: Does the S2S pipeline use the text encoder?

- Answer: In the HuggingFace implementation, for S2S, the speech encoder output is projected and fed directly to the text decoder's cross-attention. The text encoder is NOT used in the S2S forward pass.
- This means: **You might be able to remove the text encoder entirely** for S2S-only deployment, saving ~350M params.

Implementation:

```
# Check if text encoder is used in S2S
# In SeamlessM4Tv2ForSpeechToSpeech, the forward pass is:
# speech_encoder → text_decoder (cross-attending to encoder output) → t2u → vocoder
# The text_encoder is NOT invoked for S2S.

# Option A: Remove entirely (aggressive but valid for S2S-only)
if hasattr(model, 'text_encoder'):
    del model.text_encoder
    # May need to handle config and forward pass

# Option B: Prune conservatively (keep 12 of 24 layers) if it IS used
# Use same iterative method as Phase 2
```

Expected savings:

- If removable entirely: ~350M params
- If pruning 8-12 layers: ~100-150M params

Phase 4: Speech Encoder Layer Pruning (~150-200M reduction)

Paper: ShortGPT (ACL 2025), DPHuBERT (InterSpeech 2023)

Your previous angular distance scoring was reasonable but your execution had issues. Here's the improved approach:

Method: Block Influence (BI) from ShortGPT — better than angular distance:

```

def block_influence(model, calibration_data, processor):
    """
    Block Influence metric from ShortGPT (Men et al., 2024).
    Uses the hidden state transformation ratio, not just cosine distance.
    """
    layer_bi_scores = {}
    hooks = []
    layer_io = {}

    for i, layer in enumerate(model.speech_encoder.inner_layers):
        def make_hook(idx):
            def hook(module, inp, out):
                # inp[0] shape: [batch, seq, hidden]
                x = inp[0].detach().float()
                y = (out[0] if isinstance(out, tuple) else out).detach().float()

                # BI = 1 - cos_sim(X_flattened, Y_flattened)
                # Computed per-sample, averaged
                x_flat = x.reshape(x.shape[0], -1)
                y_flat = y.reshape(y.shape[0], -1)
                cos = F.cosine_similarity(x_flat, y_flat, dim=-1)

                if idx not in layer_io:
                    layer_io[idx] = []
                layer_io[idx].append((1 - cos).mean().item())
            return hook
        hooks.append(layer.register_forward_hook(make_hook(i)))

    # Run calibration
    for wav in calibration_data:
        inputs = processor(audios=wav, sampling_rate=16000, return_tensors="pt")
        inputs = {k: v.to(model.device) for k, v in inputs.items()}
        with torch.no_grad():
            model.speech_encoder(**inputs)

    for h in hooks:
        h.remove()

    # Average BI scores
    for idx in layer_io:
        layer_bi_scores[idx] = np.mean(layer_io[idx])

    return layer_bi_scores

```

Then apply iterative greedy pruning (same as Phase 2) using actual S2S quality as the metric, not just BI scores. Use BI scores only as a starting heuristic to speed up the

search.

Target: Remove 6-8 of 24 encoder layers (keep top-scoring layers, especially layer 0 and the final few layers which your previous analysis showed are critical).

Expected savings: $\sim 6-8 \text{ layers} \times \sim 25\text{M params/layer} \approx \sim 150-200\text{M params}$

Phase 5: Width Pruning with FLAP (~200-300M reduction)

Paper: FLAP (AAAI 2024), Wanda (ICLR 2024)

After removing layers, further compress the remaining layers by reducing their internal width. This is where FLAP shines — it prunes attention heads AND FFN neurons with adaptive per-layer ratios.

Key insight: FLAP doesn't just zero weights — it **physically removes columns/rows** from weight matrices, creating smaller dense matrices that run faster on GPU.

Method:

```
def flap_importance_score(weight, input_activations):
    """
    FLAP: Fluctuation-based importance score.
    Measures whether an output feature can be recovered when a column is removed.
    """
    # Baseline activation: mean output activation
    baseline = (weight @ input_activations.mean(dim=0))

    # Fluctuation: variance of output across calibration samples
    outputs = weight @ input_activations.T # [out_dim, n_samples]
    fluctuation = outputs.var(dim=1) # [out_dim]

    # Importance = fluctuation (high fluctuation → important, hard to approximate)
    return fluctuation

def structured_prune_ffn(layer, calibration_activations, prune_ratio=0.3):
    """
    Structurally prune FFN: actually remove rows/columns, not just zero them.
    """
    fc1 = layer.fc1 # [ffn_dim, hidden_dim]
    fc2 = layer.fc2 # [hidden_dim, ffn_dim]

    # Score using FLAP metric (or Wanda: |W| * ||X||)
    scores = flap_importance_score(fc1.weight.data, calibration_activations)
```

```

# Keep top (1-prune_ratio) neurons
n_keep = int(len(scores) * (1 - prune_ratio))
_, keep_indices = torch.topk(scores, n_keep)
keep_indices = keep_indices.sort().values

# STRUCTURALLY remove: create new smaller layers
new_fc1 = nn.Linear(fc1.in_features, n_keep, bias=fc1.bias is not None)
new_fc2 = nn.Linear(n_keep, fc2.out_features, bias=fc2.bias is not None)

new_fc1.weight.data = fc1.weight.data[keep_indices]
if fc1.bias is not None:
    new_fc1.bias.data = fc1.bias.data[keep_indices]
new_fc2.weight.data = fc2.weight.data[:, keep_indices]

# REPLACE (not mask) – this gives real speedup
layer.fc1 = new_fc1
layer.fc2 = new_fc2

return n_keep

def prune_attention_heads(layer, calibration_data, n_heads_remove):
    """
    Remove entire attention heads.
    Each head: (q_proj, k_proj, v_proj, out_proj) slices.
    """
    # Score heads by gradient importance or activation magnitude
    head_dim = layer.self_attn.head_dim
    n_heads = layer.self_attn.num_heads

    # Compute head importance (e.g., via attention entropy or gradient)
    head_scores = compute_head_importance(layer, calibration_data)

    # Keep top heads
    _, keep_heads = torch.topk(head_scores, n_heads - n_heads_remove)
    keep_heads = keep_heads.sort().values

    # Slice projection matrices to remove pruned heads
    for proj_name in ['q_proj', 'k_proj', 'v_proj']:
        proj = getattr(layer.self_attn, proj_name)
        keep_indices = []
        for h in keep_heads:
            keep_indices.extend(range(h * head_dim, (h + 1) * head_dim))
        new_proj = nn.Linear(proj.in_features, len(keep_indices))
        new_proj.weight.data = proj.weight.data[keep_indices]
        setattr(layer.self_attn, proj_name, new_proj)

    # out_proj: remove input dimensions for pruned heads
    out_proj = layer.self_attn.out_proj
    new_out = nn.Linear(len(keep_indices), out_proj.out_features)

```

```
new_out.weight.data = out_proj.weight.data[:, keep_indices]
layer.self_attn.out_proj = new_out
layer.self_attn.num_heads = len(keep_heads)
```

Apply to: All remaining layers in speech encoder, text decoder, and T2U. **Target:** 20-30% width reduction across all remaining layers.

Expected savings: ~200-300M params with actual inference speedup.

Phase 6: T2U Model Pruning (~50-100M reduction)

The T2U (Text-to-Unit) model has 6 transformer encoder layers + NAR frontend + 6 FeedForward Transformer layers. Apply the same iterative layer pruning:

- Try removing 2-3 layers from each stack
 - Evaluate using unit accuracy or downstream speech quality (BLEU on Whisper-transcribed output)
-

Phase 7: Recovery Fine-tuning

You already discovered the correct loss function. Use S2TT cross-entropy:

```
outputs = pruned_model(input_features=audio_features, labels=bengali_text_ids)
loss = outputs.loss # Cross-entropy on translated text tokens
```

Key improvements over your previous fine-tuning:

1. **Use more data:** FLEURS train + CoVoST2 (100K samples) + FLORES-200
2. **Train longer:** 2,000-5,000 steps minimum (not 500)
3. **Multi-language training:** Include all target language pairs simultaneously
4. **Learning rate:** Start at 1e-5, cosine decay to 1e-6
5. **LoRA for efficient training on T4:**

```
from peft import LoraConfig, get_peft_model

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
```

```
target_modules=["q_proj", "v_proj", "k_proj", "out_proj",  
               "fc1", "fc2"],  
lora_dropout=0.05,  
)  
model = get_peft_model(model, lora_config)
```

Execution Plan for Kaggle (T4 GPU, 16GB VRAM)

Session 1: Baseline + Architecture Analysis (~2 hours)

```
Cell 1: Load model, count params per component (exact breakdown)  
Cell 2: Load FLEURS eval set (en→bn, en→hi, en→zh, en→fr)  
Cell 3: Run baseline benchmark (BLEU, ChrF, RTF) – 20 samples per pair  
Cell 4: Save baseline results to Drive
```

Session 2: Vocabulary Trimming (~1 hour)

```
Cell 1: Load model + tokenizer  
Cell 2: Identify tokens used by target languages (using FLORES-200 corpus)  
Cell 3: Trim vocabulary (reduce embedding matrix)  
Cell 4: Quick sanity check – verify model still generates  
Cell 5: Benchmark trimmed model  
Cell 6: Save trimmed model to Drive
```

Session 3: Text Encoder Removal / Pruning (~2 hours)

```
Cell 1: Load trimmed model from Session 2  
Cell 2: Verify text encoder is NOT used in S2S forward pass  
Cell 3a: If not used → remove entirely  
Cell 3b: If used → iterative layer pruning (remove 8-12 layers)  
Cell 4: Benchmark  
Cell 5: Save to Drive
```

Session 4: Text Decoder Iterative Layer Pruning (~3-4 hours)

```
Cell 1: Load model from Session 3  
Cell 2: Set up FLEURS validation set for evaluation  
Cell 3: Run iterative greedy pruning (remove 1 layer at a time, evaluate)  
        Target: remove 6-8 layers
```

Metric: ChrF++ (as recommended by IWSLT 2025)
Cell 4: Benchmark after decoder pruning
Cell 5: Save to Drive

Session 5: Speech Encoder Layer Pruning (~3-4 hours)

Cell 1: Load model from Session 4
Cell 2: Compute Block Influence scores for all 24 encoder layers
Cell 3: Run iterative greedy pruning on speech encoder
Target: remove 6-8 layers
Cell 4: Benchmark
Cell 5: Save to Drive

Session 6: Width Pruning with FLAP (~2-3 hours)

Cell 1: Load model from Session 5
Cell 2: Collect calibration activations (200 samples through model)
Cell 3: Apply FLAP to all remaining FFN layers (20-30% width reduction)
Cell 4: Prune low-importance attention heads (remove 25% per layer)
Cell 5: Benchmark
Cell 6: Save to Drive

Session 7: T2U Pruning (~1-2 hours)

Cell 1: Load model from Session 6
Cell 2: Iterative layer pruning on T2U (remove 2-3 layers per stack)
Cell 3: Benchmark
Cell 4: Save to Drive

Session 8-10: Recovery Fine-tuning (~3 sessions × 4 hours)

Cell 1: Load pruned model
Cell 2: Load training data (FLEURS + CoVoST2, all target language pairs)
Cell 3: Set up LoRA (r=16, target all linear layers)
Cell 4: Train with S2TT cross-entropy loss
- Session 8: Steps 0-2000
- Session 9: Steps 2000-4000
- Session 10: Steps 4000-5000 + final benchmark
Cell 5: Save checkpoint every 500 steps to Drive

Session 11: Final Benchmark + Paper Table

Cell 1: Load final fine-tuned model

Cell 2: Full benchmark on FLEURS test (all target language pairs)

Cell 3: Compare: Baseline vs each pruning stage vs final

Cell 4: Measure: BLEU, ChrF, RTF, model size, inference memory

Cell 5: Generate paper table and figures

Expected Results Table (Projected)

Stage	Params(M)	Δ Size	BLEU	RTF
0. Baseline	2,300	—	12.2	0.24
1. Vocab trimmed	2,080	-10%	~12.0	~0.23
2. Text encoder removed/pruned	1,730	-25%	~11.5	~0.22
3. Text decoder pruned (-8L)	1,570	-32%	~9.0	~0.20
4. Speech encoder pruned (-8L)	1,370	-40%	~6.0	~0.17
5. Width pruning (FLAP 25%)	1,100	-52%	~4.0	~0.14
6. T2U pruned	1,050	-54%	~3.5	~0.13
7. After fine-tuning (5K steps)	1,050	-54%	~9-11	~0.13

Note: These are conservative estimates. The IWSLT 2025 paper achieved 97-100% quality retention at 50% compression with sufficient fine-tuning data.

Critical Implementation Notes

1. Order Matters

Do vocabulary trimming FIRST (safest, largest free gain), then layer pruning (text decoder before speech encoder), then width pruning last (most fragile).

2. Benchmark After Every Phase

Don't batch multiple pruning phases without checking. If quality collapses at any stage, you know exactly which phase to dial back.

3. Use ChrF++ for Layer Importance Evaluation

The IWSLT 2025 paper found ChrF++-guided pruning produces better models than COMET or BLEU-guided pruning.

4. Iterative > One-Shot

The key insight from IWSLT 2025: remove ONE layer, re-evaluate, then decide the next layer to remove. This is slower but dramatically better than computing importance scores once and removing all layers at once.

5. Structural Removal > Zeroing

Always physically remove rows/columns/layers. Never just zero them out. This is what gives actual speed improvement.

6. LoRA for Fine-tuning on T4

Full fine-tuning of 1B params won't fit on T4. Use LoRA ($r=16$) targeting all linear layers. This makes training feasible with ~4-6GB VRAM overhead.

7. Multi-Language Calibration and Evaluation

When computing layer importance, use calibration data from ALL your target languages, not just English. A layer that's unimportant for English might be critical for Bengali.

References

- [1] Moslem, Y. (2025). "Efficient Speech Translation through Model Compression and Knowledge Distillation." IWSLT 2025. <https://aclanthology.org/2025.iwslt->
- [2] Moslem, Y., Al Farouq, M.H., & Kelleher, J. (2025). "Iterative Layer Pruning for Efficient Translation Inference." WMT 2025. <https://aclanthology.org/2025.wmt-1.78/>
- [3] Rostami, P. & Dousti, M.J. (2024). "CULL-MT: Compression Using Language and Layer pruning for Machine Translation." arXiv:2411.06506.
- [4] An, Z. et al. (2024). "FLAP: Fluctuation-based Adaptive Structured Pruning for Large Language Models." AAAI 2024.

- [5] Sun, M. et al. (2024). "A Simple and Effective Pruning Approach for Large Language Models." (Wanda) ICLR 2024.
- [6] Ashkboos, S. et al. (2024). "SliceGPT: Compress Large Language Models by Deleting Rows and Columns." ICLR 2024.
- [7] Men, X. et al. (2024). "ShortGPT: Layers in Large Language Models are More Redundant Than You Expect." ACL 2025 Findings.
- [8] Yang, Y. et al. (2024). "LaCo: Large Language Model Pruning via Layer Collapse." EMNLP 2024 Findings.
- [9] Asahi, O. et al. (2023). "An Efficient Multilingual Language Model Compression through Vocabulary Trimming." EMNLP 2023 Findings.
GitHub: <https://github.com/asahi417/lm-vocab-trimmer>
- [10] Peng, Y. et al. (2023). "DPHuBERT: Joint Distillation and Pruning of Self-Supervised Speech Models." InterSpeech 2023.
- [11] Meta AI. "SeamlessM4T On-Device Models."
https://github.com/facebookresearch/seamless_communication/blob/main/docs/m4t
- [12] ymoslem/Model-Compression – GitHub repository for IWSLT/WMT 2025 experiments
<https://github.com/ymoslem/Model-Compression>

